



Markers! Documentation

Live version: https://docs.google.com/document/d/1GiWIA_B66mdiAt018q7cP5wH7mihRbyi4IZi4U0WPq0/

About Markers!

Markers! is the easiest way to add UI screen markers to your game. Whether it is quest markers, objective indicators or waypoint icons, this tool makes it quick and intuitive. Plug and play, fully customizable and perfect for both 2D and 3D games. No setup hassle. Just mark it.

Supported Unity Render Pipelines

Markers! fully supports Built-in (BIRP), Universal Render Pipeline (URP) and High-Definition Render Pipeline (HDRP).

Supported Unity Versions

Markers! is compatible with Unity 2021 LTS, 2022 LTS, as well as Unity 6 and 6.1.

Note: While our Markers! is likely to work with most newer alpha and beta versions of Unity, these versions are not officially supported.

Supported Platforms

Markers! is supported on every platform that Unity supports.

Support

We believe in going the extra mile for supporting our assets, if you have any questions, please contact us for support through [e-mail](#) or live on our [discord](#) server, during our office hours between 10:00 and 18:00 CEST.

The Naked Dev Tools

Upscaling Tools

Every upscaler has its own strengths and weaknesses. Some require specific hardware, while others are designed for slower or older devices.

Below is a list of all other upscalers we offer for Unity:

[FSR 3 - Upscaling for Unity](#)

FSR 3 works on most platforms and hardware, making it the top choice for compatibility. However, its visual quality is slightly below that of DLSS or XeSS.

[XeSS 2 - Upscaling for Unity](#)

XeSS 2 runs on Windows x64 with DX11/DX12 and supports Intel, AMD, and Nvidia GPUs. Its visual quality surpasses FSR 3 and can even rival DLSS in some cases.

[DLSS - Upscaling for Unity](#)

DLSS supports Windows x64 with DX11/DX12 and works only on Nvidia RTX GPUs (20-series and newer). Its visual quality is higher than FSR 3 and, in some cases, better than XeSS.

[SGSR 1 - Upscaling for Unity](#)

SGSR 1 runs on nearly all platforms and hardware. Its visual quality is lower than other upscalers, but it's significantly faster, making it ideal for high-DPI platforms like mobile devices.

[SGSR 2 - Upscaling for Unity](#)

SGSR 2 is supported on nearly all platforms and hardware. It delivers much better visual quality than SGSR 1 with only slightly higher GPU cost. A fragment shader version is also available, enabling support for even older hardware, perfect for mobile devices like Android, iOS, and Nintendo Switch.

Graphics Tools

[CACAO - Ambient Occlusion for Unity](#)

Coming Soon! - CACAO is an ambient occlusion technique, created for AAA games, to produce the best possible ambient occlusion visuals while also offering the best performance possible.

Game-dev Tools

[Markers!](#)

Markers! is the easiest way to add UI screen markers to your game. Whether it is quest markers, objective indicators or waypoint icons, this tool makes it quick and intuitive. Plug and play, fully customizable and perfect for both 2D and 3D games. No setup hassle. Just mark it.

About Markers!	2
Supported Unity Render Pipelines	2
Supported Unity Versions	2
Supported Platforms	2
Support	2
The Naked Dev Tools	3
Upscaling Tools	3
FSR 3 - Upscaling for Unity	3
XeSS 2 - Upscaling for Unity	3
DLSS - Upscaling for Unity	3
SGSR 1 - Upscaling for Unity	3
SGSR 2 - Upscaling for Unity	3
Graphics Tools	3
CACAO - Ambient Occlusion for Unity	3
Game-dev Tools	3
Markers!	3
Quick Start	5
Important Information	6
Markers! uses Unity's Canvas for rendering overlay	6
Demo Scenes	7
Inspector	8
Marker Settings	8
Marker Manager Prefab	9
Marker Template Prefab	10
Editing or creating your own Marker Template Prefab	10
Public API	12
Marker Manager	12
public bool SetCamera(Camera value)	12
public bool SetPlayer(Transform value)	12
public bool DestroyAllMarkers()	12
public bool ShowCanvas(bool value)	12
public bool HideCanvas(bool value)	12
Marker Component	12
public bool SetMarkerType(int value)	12
public bool SetMarkerName(string value)	12
Uninstall	13
Support	13

Quick Start

This chapter is meant to help you integrate Markers! into your project as quickly as possible. However, we strongly recommend reading the rest of the documentation, especially the chapter titled [Important Information](#), to fully understand how the asset works.

Step 1: Import Markers! from the Unity Package Manager.

Step 2: Add the **Marker Manager** prefab, located in the Prefabs folder to your scene.

Step 3: On the **Marker Manager**, drag in your Main Camera in the **Camera** slot.

Step 4: On the **Marker Manager**, drag in the root GameObject of your player in the **Player** slot.

Step 5: Add the **Marker Template** prefab, located in the Prefabs folder to your scene, and position it in front of your camera.



Hit play!

Note: The root GameObject can also be your Main Camera.

Important Information

Markers! works on a simple **No Coding** design principle, when it is used in this fashion there is little to no coding involved!

If you want to add a marker, just add the **MarkerComponent** to a GameObject in your scene and voila, now you have an on-screen Marker.

When you want this marker to be temporarily disabled, disable the **MarkerComponent** (or GameObject), this fades out the on-screen marker. Simply enable the **MarkerComponent** or its GameObject again to enable the on-screen marker.

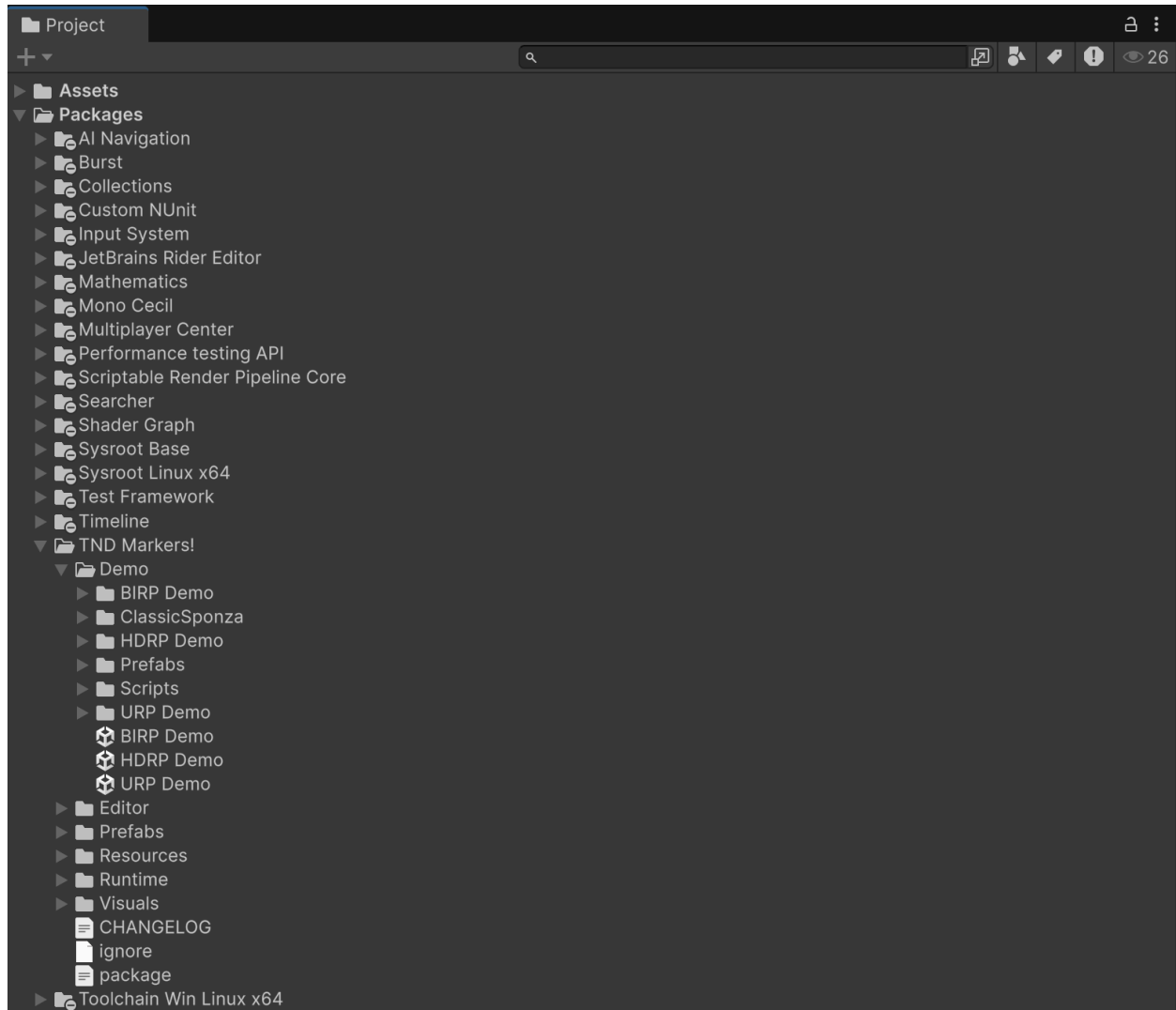
Destroying the **MarkerComponent** will remove the on-screen marker permanently.

Markers! uses Unity's Canvas for rendering overlay

Demo Scenes

We've included demo scenes for each render pipeline, allowing you to try-out Markers! before integrating it into your own project.

You can find them under: **"Packages/TND Markers!/Demo/"**

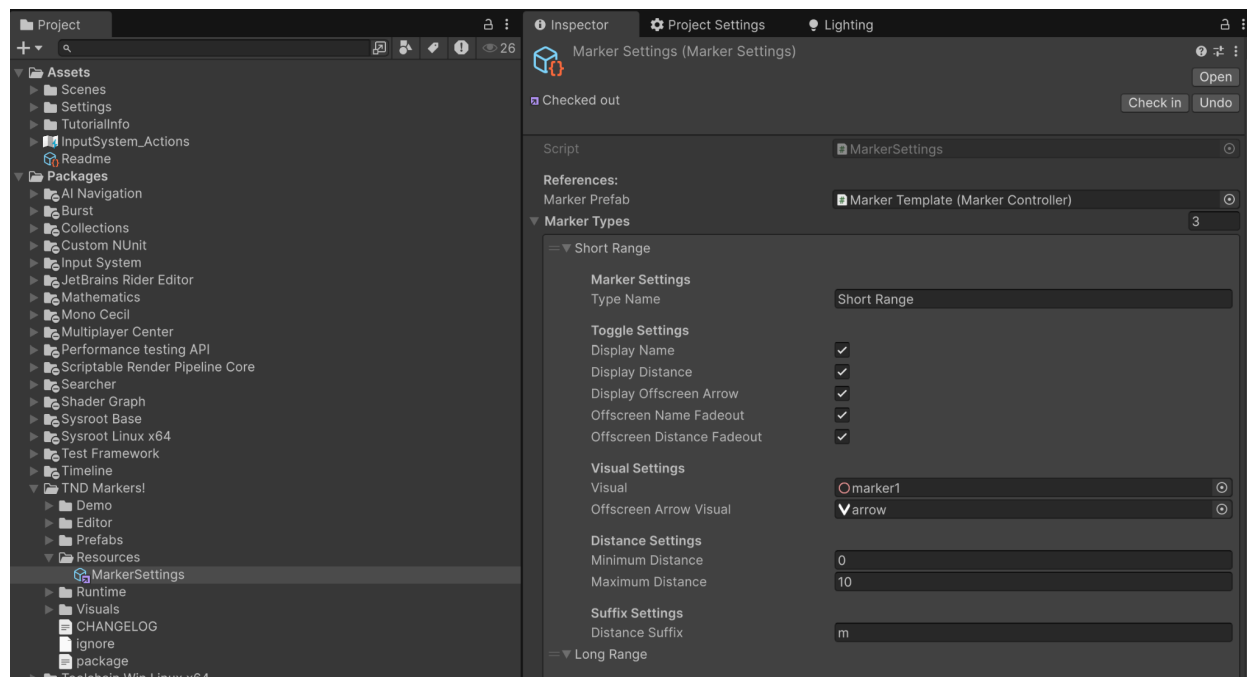


Inspector

This chapter explains all the Inspector settings of Markers!

Marker Settings

Use this Scriptable Object to set up as many different types of markers you need for your project.



Type Name: This will be converted into an enum, to be selected on the Marker Component.

Display Name: The text that will be displayed on the `_thisMarkerController`.

Display Distance: The distance, from player to marker, that will be displayed on the marker.

Display Offscreen Arrow: Toggle an additional 2D arrow sprite when the Marker is off-screen.

Offscreen Name Fadeout: Toggle displaying the marker name when off-screen.

Offscreen Distance Fadeout: Toggle displaying the distance when off-screen.

Visual: The 2D marker visual.

Offscreen Arrow Visual: The 2D marker off-screen arrow visual.

Minimum Distance: The minimum distance for the marker to show.

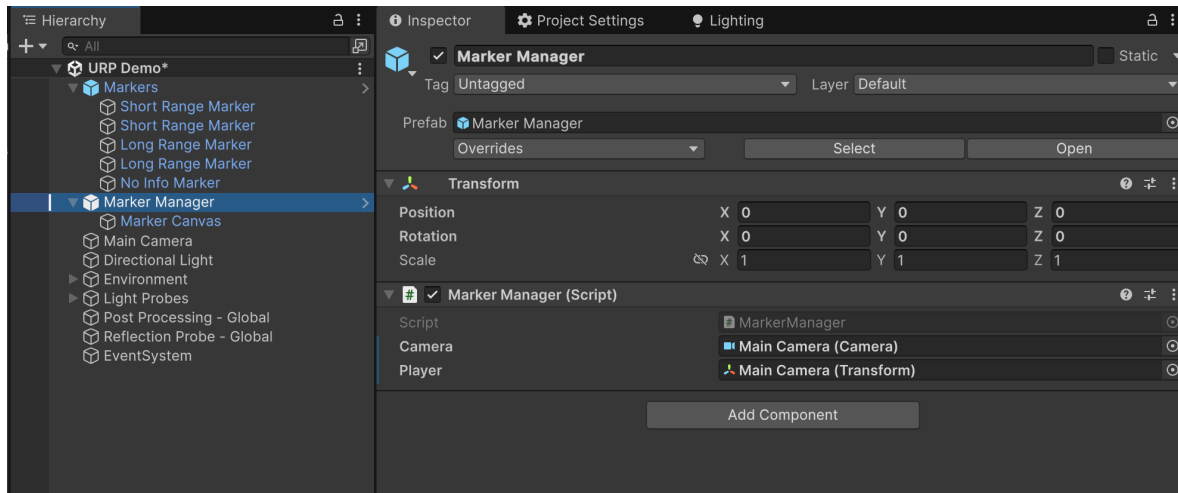
Maximum Distance: The maximum distance for the marker to show.

Distance Suffix: A helpful extra appendix added to the distance text.

Note: These values should not be modified during runtime.

Marker Manager Prefab

The Marker Manager needs the reference to your Main Camera and Player GameObject.



It requires a reference to the Main Camera so that it can calculate the World-to-Screen-Point of the Marker Component. The Player Transform is used to calculate the distance between it and the marker for displaying the distance.

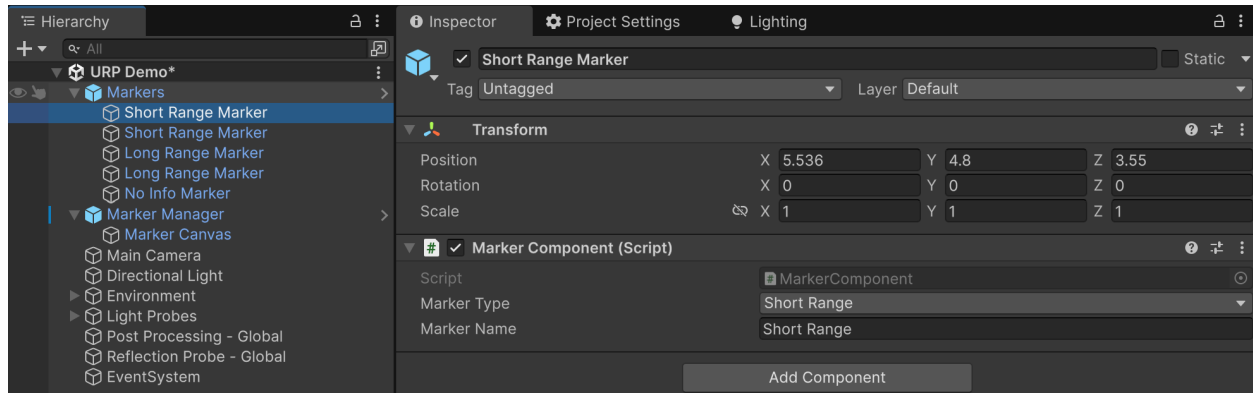


Note: In many situations, selecting the Main Camera as Player Transform is perfectly fine.

Note2: These values can be changed during runtime by using the public API.

Marker Template Prefab

The Marker Template Prefab can be used to instantiate Markers. Place them on the location you want them, or parent them to whatever GameObject you want, the Markers will update their location automatically.



Marker Type: Select which of your preset Marker Types this specific Marker is.

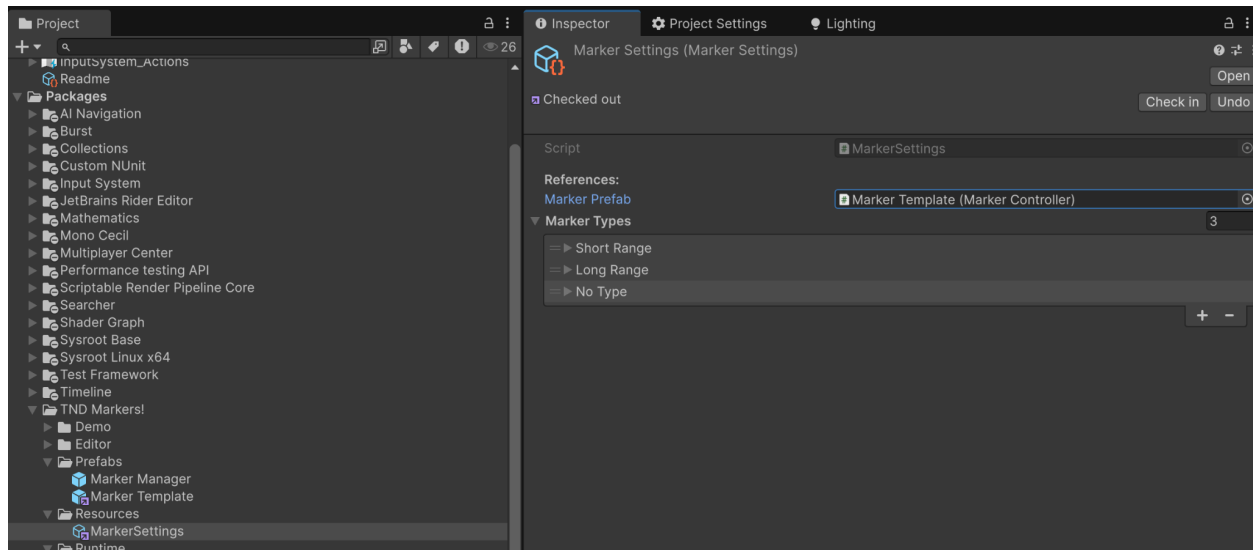
Marker Name: Set what name the Marker should display on screen.

Note: The **Marker Type** and **Marker Name** values can be changed during runtime by using the public API.

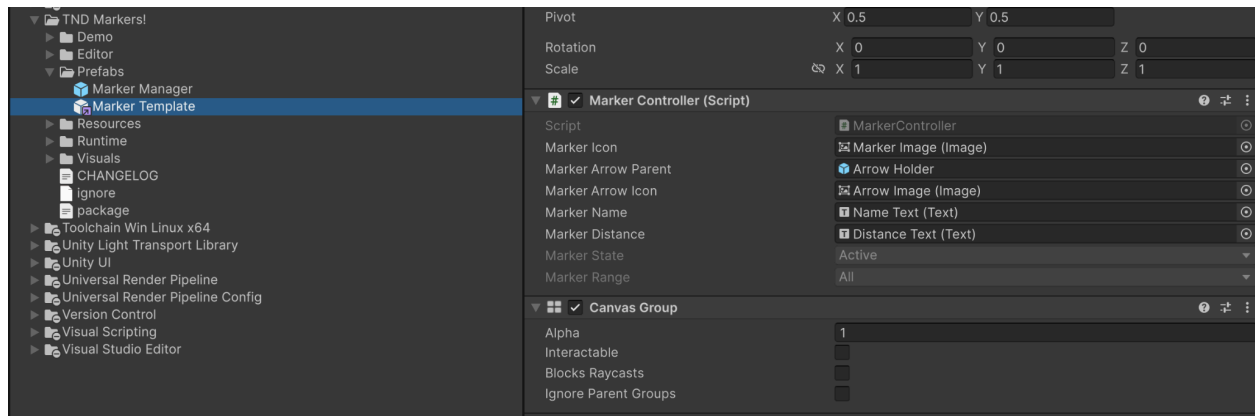
Editing or creating your own Marker Template Prefab

You are of course free to edit or create your own Marker Template Prefab.

Just make sure you update the **Marker Template Prefab** in the MarkerSettings.



Also make sure that you keep the references in the **MarkerController** script alive. Breaking these references will not throw any errors, but may result in certain visual Marker features not working any longer.



Public API

To use our public API, simply create a script that references the `MarkerComponent` and/or `MarkerManager` components and call the public methods as described in this chapter.

Marker Manager

public bool SetCamera(*Camera* value)

Set the Camera reference.

public bool SetPlayer(*Transform* value)

Set the Player reference.

public bool DestroyAllMarkers()

Destroys all markers.

public bool ShowCanvas(*bool* value)

Fades the Marker Canvas in.

public bool HideCanvas(*bool* value)

Fades the Marker Canvas out.

Marker Component

public bool SetMarkerType(*int* value)

Set the type of the Marker, use the integer value of the index of the **Marker Types** list in the **MarkerSettings**.

public bool SetMarkerName(*string* value)

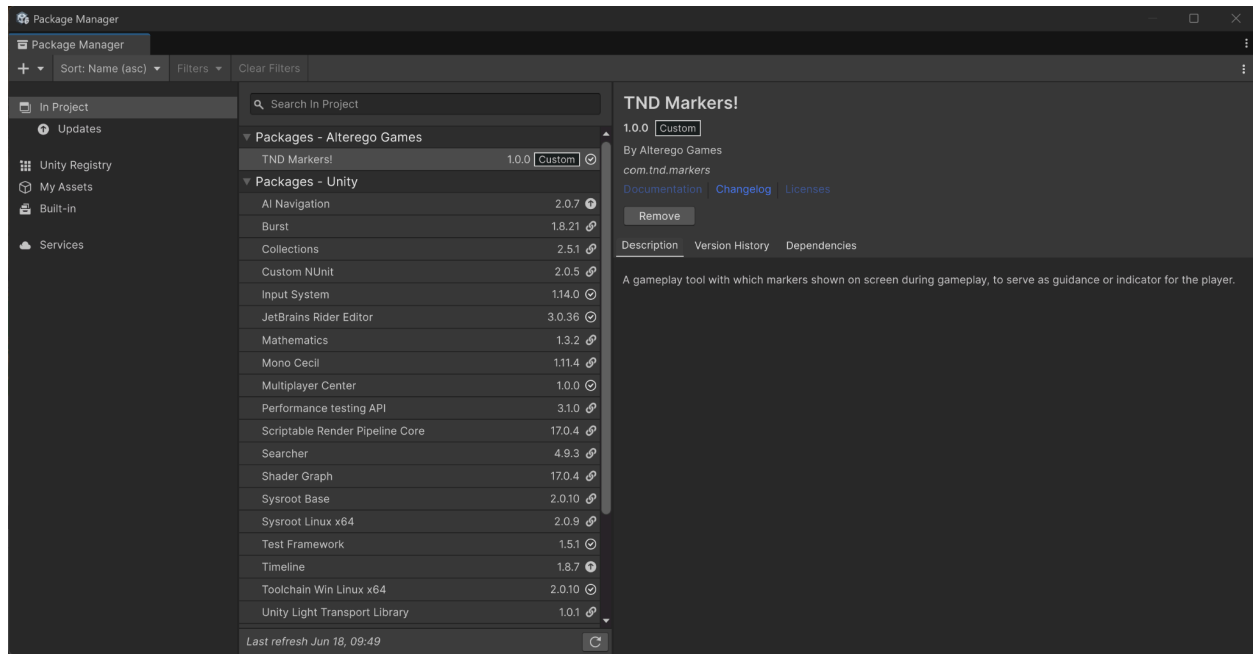
Set the Marker name, which is visualized on the UI marker.

Uninstall

Step 1: Open Unity's Package Manager

Step 2: Select the **TND Markers!** package.

Step 3: Click **Remove**.



Support

If you encounter any issues, you can contact us by emailing to info@thenakeddev.com, or by joining our [discord](#).